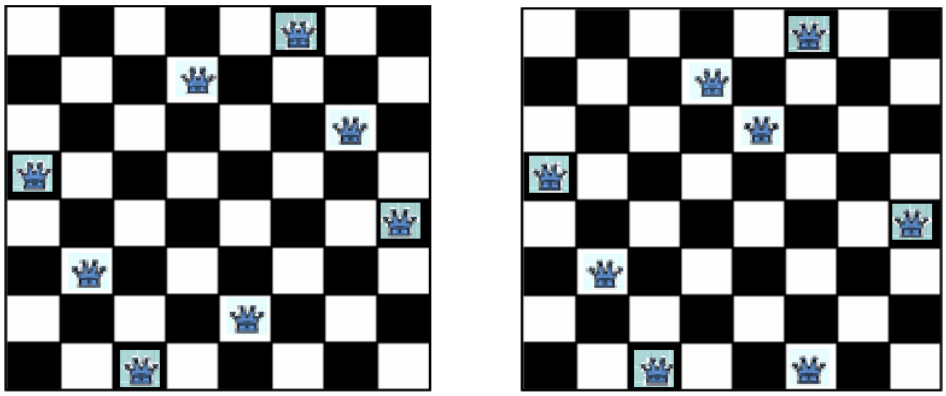
Lab 1: Problem Solving by Searching: N Queens Problem

Problem Statement: The problem is to place 8 queens on a chessboard so that no two queens are in the same row, column or diagonal.

The picture below on the left shows a solution of the 8-queens problem. The picture on the right is not a correct solution, because some of the queens are attacking each other.



Problem Formulation:

- **State Space:** Any arrangement of **k** queens in the first k rows such that none are attacked
- **Initial state:** 0 queens on the board
- **Successor function:** Add a queen to the $(k+1)^{th}$ row so that none are attacked.
- **Goal test:** 8 queens on the board, none are attacked

Source Code:

```
import pprint

def isSafe(board, x, y, n):
  #Checking whether the column is filled
  for row in range(x):
    if(board[row][y] == 'Q'):
      return False

  #Checking for top left diagonals are filled
  row = x
  col = y
  while(row>=0 and col>=0):
    if(board[row][col] == 'Q'):
      return False
    row -= 1
    col -= 1

  #Checking for top right diagonals are filled
  row = x
  col = y
  while(row>=0 and col<n):
    if(board[row][col] == 'Q'):
      return False
    row -= 1
    col += 1
  #return True if all the aforementioned tests is passed
  return True

def nQueen(board, x, n):
  #if we have successfully placed n queens return True
  if(x>=n):
    return True
  #iterate through columns for each row
  for col in range(n):
    #if the particular position is safe then place that queen
    if(isSafe(board,x,col,n)):
      board[x][col] = 'Q'
      #if the next queen cannot be placed then backtrack
      if(nQueen(board,x+1,n)):
        return True
      board[x][col] = ' '
  return False
```

```
n = int(input("Enter number of Q "))
board = [[' ']*n for i in range(n)]
if(nQueen(board,0,n)):
  pprint.pprint(board)
else:
  print("No Solution")
```

Output:

```
pukarkarki@macbookpro Lab1 % python3 Lab1AI.py
Enter number of Q 8
[['Q', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
 [' ', ' ', ' ', ' ', 'Q', ' ', ' ', ' '],
 [' ', ' ', ' ', ' ', ' ', ' ', ' ', 'Q'],
 [' ', ' ', ' ', ' ', ' ', 'Q', ' ', ' '],
 [' ', ' ', 'Q', ' ', ' ', ' ', ' ', ' '],
 [' ', ' ', ' ', ' ', ' ', ' ', 'Q', ' '],
 [' ', 'Q', ' ', ' ', ' ', ' ', ' ', ' '],
 [' ', ' ', ' ', 'Q', ' ', ' ', ' ', ' ']]
```

Task 1: Goat, Wolf and Cabbage Problem

Problem Statement

A farmer has a goat, a wolf and a cabbage on the west side of a river. He wants to transport all three to the east side using a boat that can carry only the farmer and one additional item. The following constraints must always be satisfied:

• The wolf cannot be left alone with the goat.

• The goat cannot be left alone with the cabbage.

Tasks

1. Formulate the problem as a state-space search problem.
2. Define:

• State representation

• Initial state

• Goal state

• Valid operators/actions

3. Write a Python program to solve the problem using a search algorithm.

Expected Output

The program should print the sequence of moves required to safely transport all items across the river.

Task 2: Water Jug Problem

Problem Statement

You are given two water jugs:

• Jug A = 4 liters

• Jug B = 3 liters

Initially, both jugs are empty.

The objective is to obtain exactly 2 liters of water in Jug A.

Tasks

1. Represent the problem as a state-space search problem.
2. Define:

• State representation

• Initial state

• Goal state

• Valid actions

3. Write a Python program that solves the problem using Breadth First Search (BFS).

4. Print the shortest solution path from the initial state to the goal state.

Possible Actions

• Fill Jug A

• Fill Jug B

• Empty Jug A

• Empty Jug B

• Pour water from Jug A to Jug B

• Pour water from Jug B to Jug A

Expected Output

The program should display the sequence of states leading to a state where Jug A contains exactly 2 liters of water.

Task 3: Robot Navigation Problem

Problem Statement

A robot is placed in a 5 ×5 grid and must move from the start position (S) to the goal position (G) while avoiding obstacles (X).

Grid Layout

S . . X .

. X . X .

. X . . .

. . X X .

. . . . G

Tasks

1. Represent the grid as a search space.
2. Define:
• State representation
• Initial state
• Goal state
• Valid actions
3. Write a Python program to find a path from the start cell to the goal cell using BFS.
4. Print the shortest path found by the algorithm.

Valid Actions

• Move Up
• Move Down
• Move Left
• Move Right

Expected Output

The program should display:
• The sequence of coordinates visited